Task 1 :- Code Understanding :

Data structure : Array - based stack (LIFO - Last In, First Out).

Variables used :

Size (macro) : 5 (Max Capacity)

Stack [SIZE] : Integer array for Storage.

top : Integer tracking the highest Index (Initial : -1).

item : Local Integer in main() & push().

i : Loop variable in display().

Operations & Functions :-

Push (int item) : Checks overflow (top == SIZE - 1), adds item, increments top.

pop() : checks underflow (top == -1), returns item, decrements top.

display() : loops i from 0 to top to print Contents.

Task 4 : Output Prediction :

Predicted Output :

pushed : 10, pushed : 20, pushed : 30

Stack : 10 20 30

Popped : 30

pushed : 40, pushed : 50, pushed : 60

Stack Overflow

Stack : 10 20 40 50 60

Popped : 60, popped : 50, popped : 40, popped : 20, popped : 10.

Stack is empty.

Stack underflow.

Popped : -1.

→ Final Stack State : Empty.

→ Final top value : -1.

→ Overflow Condition : Reached at push(70) because top is 4 (maximum index).

→ Underflow Condition : Reached at the final pop() because all items are gone (top = -1).

TRACE TABLE :

STEP	OPERATION	TOP	ITEM	i	Stack Contents	OUTPUT / PRINT
1	Initial State	-1	-	-	Empty	-
2	push(10)	0	10	-	[10]	pushed : 10
3	push(20)	1	20	-	[10, 20]	pushed: 20
4	push(30)	2	30	-	[10, 20, 30]	pushed: 30
5	display()	2	-	0,1,2	[10, 20, 30]	Stack : 10 20 30
6	item = pop()	1	30	-	[10, 20]	popped : 30
7	push (40)	2	40	-	[10, 20, 40]	pushed: 40
8	push (50)	3	50	-	[10, 20, 40, 50]	pushed : 50
9	push (60)	4	60	-	[10, 20, 40, 50, 60]	pushed : 60
10	push(70)	4	70	-	[10, 20, 40, 50, 60]	Stack Overflow.
11	display()	4	-	0,1,2,3,4	[10, 20, 40, 50, 60]	Stack : 10 20 40 50 60
12	item = pop()	3	60	-	[10, 20, 40, 50]	popped : 60
13	item = pop()	2	50	-	[10, 20, 40]	popped : 50
14	item = pop()	1	40	-	[10, 20]	popped : 40
15	item = pop()	0	20	-	[10]	popped : 20
16	item = pop()	-1	10	-	Empty	popped : 10
17	display()	-1	-	-	Empty	Stack is Empty
18	item = pop()	-1	-1	-	Empty	Stack Underflow. Popped : -1.

Analysis & Testing.

What - if Analysis :-

Modification :- change #define SIZE 5 to # define SIZE 6.

Original Execution: push (70) results in "Stack overflow" because top == 4 triggers if (4 == 5-1).

Modified Execution : push (70) succeeds. top becomes 5, & "pushed : 70" is printed because 4 == 6-1 is false.

Impact : The final pop() in Main() will no longer cause an underflow, as there is an extra element (70) to remove.

Error Analysis :-

Erroneous Statement : return -1; inside the pop() function during underflow.

Reason: The main function assumes -1 is a valid process (i.e) popped item & prints "Popped : -1". Immediately after printing "Stack underflow", which is logically misleading. -1 could also be a valid stack Value.

Correction : Pass items by reference using pointers.

Change to : bool pop (int *popped-item) returning false on underflow, and checking the boolean in main().

Verification & test Cases

Verification :- Manual trace results perfectly match the executed Compiler output. no deviations observed.

Additional test Cases :-

Test case 1 (Immediate underflow): Call pop() as the first Statement in main().
Expected : "Stack Underflow", returns -1. Actual : Matches.

Test Case 2 (Empty Display): Call display() as the first Statement. Expected : "Stack is Empty". Actual : Matches.

Test Case 3 (Boundary check): Push 5 elements, then push a 6th.
Expected : 5 success messages, 1 "Stack Overflow". Actual : Matches.